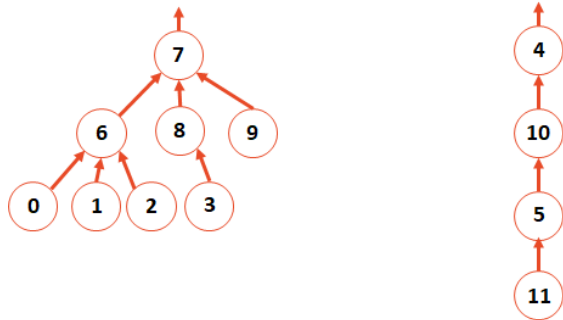


Building a Smart Union Function



The implementation of this visual model is the following:

6	6	6	8	-1	10	7	-1	7	7	4	5
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

Strategy #1: Union by Height

Idea: Keep the height of the tree as small as possible!

Metadata at Root:

After `union( 4, 7 )`:

6	6	6	8		10	7		7	7	4	5
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

Strategy #2: Union by Size

Idea: Minimize the number of nodes that increase in height.
(Observe that the tree we union have all their nodes gain in height.)

Metadata at Root:

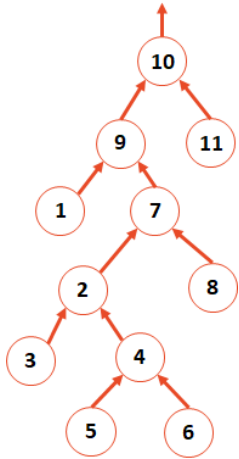
After `union( 4, 7 )`:

6	6	6	8		10	7		7	7	4	5
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

Smart Union Implementation:

DisjointSets.cpp (partial)	
1	<code>void DisjointSets::unionBySize(int root1, int root2) {</code>
2	<code> int newSize = arr_[root1] + arr_[root2];</code>
3	<code></code>
4	<code> // If arr_[root1] is less than (more negative), it is the larger</code>
5	<code> // set; we union the smaller set, root2, with root1.</code>
6	<code> if (arr_[root1] < arr_[root2]) {</code>
7	<code> arr_[root2] = root1;</code>
8	<code> arr_[root1] = newSize;</code>
9	<code> }</code>
10	<code></code>
11	<code> // Otherwise, do the opposite:</code>
12	<code> else {</code>
13	<code> arr_[root1] = root2;</code>
14	<code> arr_[root2] = newSize;</code>
15	<code> }</code>
16	<code>}</code>

Path Compression:



Disjoint Sets Analysis

The **iterated log** function:

The number of times you can take a log of a number.

$\log^*(n) =$

$$\begin{aligned} &0, n \leq 1 \\ &1 + \log^*(\log(n)), n > 1 \end{aligned}$$

What is $\lg^*(2^{65536})$?

In an Disjoint Sets implemented with smart **unions** and path compression on **find**:

Any sequence of **m union** and **find** operations result in the worse case running time of $O(\text{_____})$,
where **n** is the number of items in the Disjoint Sets.

In Review: Data Structures

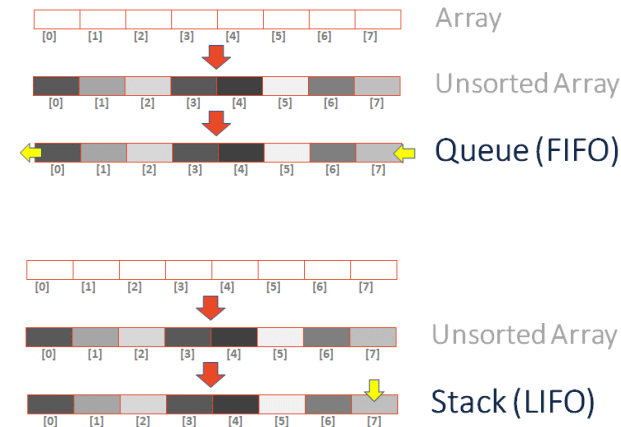
Array

- Sorted Array
- Unsorted Array
- Stacks
- Queues
- Hashing
- Heaps
 - Priority Queues
- UpTrees
- Disjoint Sets

List

- Doubly Linked List
- Skip List
- Trees
 - BTree
 - Binary Tree
 - Huffman Encoding
 - kd-Tree
 - AVL Tree

Array



CS 225 – Things To Be Doing:

1. lab_heaps due this Sunday (May 3) midnight